**BỘ GIÁO DỤC VÀ ĐÀO TẠO**
**TRƯỜNG ĐẠI HỌC [TÊN TRƯỜNG]**
**KHOA CÔNG NGHỆ THÔNG TIN**

———————— ✦ ————————

# BÁO CÁO ĐỒ ÁN MÔN HỌC

# ĐIỆN TOÁN ĐÁM MÂY

**ĐỀ TÀI**

## DESIGN AND DEPLOYMENT OF A SPRING BOOT CONTAINERIZED CLOUD FILE STORAGE PLATFORM BASED ON AWS S3-COMPATIBLE ARCHITECTURE WITH AN AUTOMATED CI/CD PIPELINE

*(Thiết kế và triển khai nền tảng lưu trữ file đám mây dạng container bằng Spring Boot theo kiến trúc tương thích AWS S3, tích hợp quy trình CI/CD tự động)*

**Sản phẩm: NIMBUS DRIVE**

| | |
|---|---|
| **Giảng viên hướng dẫn:** | [Họ tên giảng viên] |
| **Sinh viên thực hiện:** | [Họ tên sinh viên] |
| **Mã số sinh viên:** | [MSSV] |
| **Lớp:** | [Lớp học phần] |

**[Thành phố], tháng 11 năm 2026**

# LỜI CẢM ƠN

Em xin chân thành cảm ơn [giảng viên hướng dẫn] đã tận tình giảng dạy học phần Điện toán đám mây và định hướng cho em trong suốt quá trình thực hiện đồ án. Những kiến thức về mô hình dịch vụ đám mây, lưu trữ đối tượng, container hóa và tự động hóa triển khai là nền tảng để em xây dựng sản phẩm Nimbus Drive.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Em rất mong nhận được góp ý của thầy/cô để hoàn thiện sản phẩm hơn.

*Sinh viên thực hiện*

**[Họ tên sinh viên]**

# MỤC LỤC

LỜI CẢM ƠN ..... 2
MỤC LỤC ..... 3
DANH MỤC HÌNH ẢNH ..... 5
DANH MỤC TỪ VIẾT TẮT ..... 7
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI ..... 8
1.1. Lý do chọn đề tài ..... 8
1.2. Mục tiêu ..... 8
1.3. Phạm vi và đối tượng ..... 8
1.4. Phương pháp thực hiện ..... 9
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT ..... 10
2.1. Điện toán đám mây ..... 10
2.2. Lưu trữ đối tượng và giao thức Amazon S3 ..... 10
2.3. Container hóa với Docker ..... 10
2.4. Tích hợp và triển khai liên tục (CI/CD) ..... 11
2.5. Hạ tầng dưới dạng mã (Infrastructure as Code) ..... 11
2.6. Các công nghệ phía ứng dụng ..... 11
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG ..... 12
3.1. Yêu cầu chức năng ..... 12
3.2. Yêu cầu phi chức năng ..... 12
3.3. Kiến trúc tổng thể ..... 13
3.4. Kiến trúc phần mềm ..... 14
3.5. Thiết kế cơ sở dữ liệu ..... 14
3.6. Thiết kế API ..... 15
3.7. Luồng xử lý upload và tải qua presigned URL ..... 16
3.8. Thiết kế bảo mật ..... 16
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG ..... 18
4.1. Cấu trúc mã nguồn ..... 18
4.2. Tầng lưu trữ tương thích S3 ..... 18

4.3. Giao diện người dùng ..... 19
4.4. Container hóa ..... 19
4.4.1. Dockerfile nhiều giai đoạn ..... 19
4.4.2. Docker Compose ..... 19
4.5. Pipeline CI/CD với GitHub Actions ..... 20
4.6. Hạ tầng AWS bằng Terraform ..... 21
4.7. Giám sát ..... 21
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ ..... 22
5.1. Giao diện ứng dụng ..... 22
5.2. Kiểm thử ..... 28
5.3. Đánh giá ..... 28
Ưu điểm ..... 28
Hạn chế ..... 29
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN ..... 30
6.1. Kết luận ..... 30
6.2. Hướng phát triển ..... 30
TÀI LIỆU THAM KHẢO ..... 31
PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY ..... 32

# DANH MỤC HÌNH ẢNH

Bảng 0.1. Các từ viết tắt ..... 7
Bảng 2.1. Các mô hình dịch vụ và vị trí của Nimbus Drive ..... 10
Bảng 2.2. Công nghệ sử dụng ..... 11
Hình 3.1. Sơ đồ use case tổng quát ..... 12
Bảng 3.1. Yêu cầu chức năng ..... 12
Hình 3.2. Kiến trúc tổng thể hệ thống Nimbus Drive ..... 13
Hình 3.3. Kiến trúc phân lớp và mẫu Ports & Adapters cho tầng lưu trữ ..... 14
Hình 3.4. Lược đồ quan hệ thực thể (ERD) ..... 14
Bảng 3.2. Mô tả các bảng ..... 14
Bảng 3.3. Các endpoint REST chính ..... 15
Hình 3.5. Sơ đồ tuần tự upload file và tải trực tiếp bằng presigned URL ..... 16
Bảng 3.4. Các biện pháp bảo mật ..... 16
Bảng 4.1. Các service trong docker-compose.yml ..... 19
Hình 4.1. Pipeline CI/CD tự động ..... 20
Bảng 4.2. Các job trong pipeline ..... 20
Hình 4.2. Kiến trúc triển khai trên AWS ..... 21
Bảng 4.3. Tài nguyên Terraform tạo ra ..... 21
Hình 5.1. Trang đăng nhập / đăng ký ..... 22
Hình 5.2. Trang quản lý file – chế độ lưới ..... 22
Hình 5.3. Thư mục ảnh với ảnh thu nhỏ ..... 23
Hình 5.4. Xem trước ảnh trực tiếp ..... 23
Hình 5.5. Chế độ danh sách và breadcrumb thư mục lồng nhau ..... 24
Hình 5.6. Menu thao tác trên file ..... 24
Hình 5.7. Hộp thoại tạo link chia sẻ có thời hạn ..... 25
Hình 5.8. Quản lý các link đã chia sẻ ..... 25
Hình 5.9. Giao diện tối (dark mode) ..... 26
Hình 5.10. Trang tải file công khai cho khách ..... 26
Hình 5.11. Tài liệu API Swagger UI ..... 27

Hình 5.12. Giao diện trên điện thoại .....27
Bảng 5.1. Kết quả kiểm thử tự động (mvn verify) .....28
Bảng 5.2. Đối chiếu mục tiêu .....28
Bảng 6.1. Đề xuất phát triển tiếp .....30

# DANH MỤC TỪ VIẾT TẮT

*Bảng 0.1. Các từ viết tắt*

| Viết tắt | Ý nghĩa |
|---|---|
| API | Application Programming Interface – giao diện lập trình ứng dụng |
| AWS | Amazon Web Services |
| S3 | Amazon Simple Storage Service – dịch vụ lưu trữ đối tượng |
| CI/CD | Continuous Integration / Continuous Delivery (Deployment) |
| IaaS / PaaS / SaaS | Infrastructure / Platform / Software as a Service |
| IaC | Infrastructure as Code – hạ tầng dưới dạng mã |
| JWT | JSON Web Token |
| JPA | Jakarta Persistence API |
| IAM | Identity and Access Management |
| EC2 | Amazon Elastic Compute Cloud |
| GHCR | GitHub Container Registry |
| SigV4 | AWS Signature Version 4 – cơ chế ký request của S3 |
| CSP | Content Security Policy |
| XSS | Cross-Site Scripting |

# CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

## 1.1. Lý do chọn đề tài

Nhu cầu lưu trữ và chia sẻ dữ liệu trực tuyến tăng mạnh cùng với sự phổ biến của làm việc từ xa và học tập trực tuyến. Các dịch vụ như Google Drive, Dropbox hay OneDrive đều được xây dựng trên nền tảng **lưu trữ đối tượng (object storage)** – loại lưu trữ có khả năng mở rộng gần như vô hạn, độ bền dữ liệu cao và chi phí thấp. Trong đó, giao thức **Amazon S3** đã trở thành chuẩn de-facto: hầu hết nhà cung cấp (Google Cloud Storage, Cloudflare R2, DigitalOcean Spaces, Ceph, SeaweedFS, MinIO…) đều cung cấp API tương thích S3.

Bên cạnh đó, phương thức phát triển phần mềm hiện đại đòi hỏi ứng dụng được **đóng gói thành container** và **triển khai tự động** qua pipeline CI/CD để giảm lỗi thủ công, rút ngắn thời gian đưa tính năng tới người dùng. Đề tài kết hợp cả ba trụ cột của điện toán đám mây: dịch vụ lưu trữ đám mây, container hóa và tự động hóa vận hành (DevOps).

## 1.2. Mục tiêu

- Thiết kế và xây dựng ứng dụng web lưu trữ file **Nimbus Drive** bằng Spring Boot 3, cho phép người dùng đăng ký, tải lên, tổ chức thư mục, xem trước, tìm kiếm và chia sẻ file.
- Tách biệt **metadata** (PostgreSQL) và **nội dung file** (object storage) theo kiến trúc tương thích S3; ứng dụng chạy được trên Amazon S3 thật hoặc storage tự host mà không đổi mã nguồn.
- Đóng gói toàn bộ hệ thống bằng **Docker / Docker Compose**; mô tả hạ tầng AWS bằng **Terraform**.
- Xây dựng pipeline **GitHub Actions** tự động: build → kiểm thử → build image → quét lỗ hổng → đẩy registry → triển khai → smoke test.
- Đảm bảo các yêu cầu phi chức năng: bảo mật, khả năng giám sát (observability), dễ mở rộng.

## 1.3. Phạm vi và đối tượng

Đối tượng sử dụng là cá nhân hoặc nhóm nhỏ cần kho lưu trữ riêng (self-hosted) hoặc triển khai trên AWS. Phạm vi đồ án gồm: backend REST API, giao diện web SPA, cơ sở dữ liệu, lớp lưu trữ đối tượng, đóng gói container, hạ tầng dưới dạng mã và pipeline CI/CD. Không bao gồm ứng dụng di động native, đồng bộ hai chiều kiểu desktop client và thanh toán.

### 1.4. Phương pháp thực hiện

- Nghiên cứu tài liệu chính thức: AWS S3 Developer Guide, Spring Boot Reference, Docker, GitHub Actions, Terraform AWS Provider.
- Phân tích yêu cầu → thiết kế kiến trúc phân lớp kết hợp mẫu **Ports & Adapters** cho tầng lưu trữ.
- Phát triển lặp, kiểm thử tự động (unit test, API test với MockMvc, integration test với Testcontainers).
- Tự động hóa build/deploy, đo đạc và giám sát bằng Spring Boot Actuator, Prometheus và Grafana.

# CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

## 2.1. Điện toán đám mây

Theo định nghĩa của NIST (SP 800-145), điện toán đám mây là mô hình cho phép truy cập qua mạng, theo yêu cầu, tới một tập tài nguyên tính toán dùng chung có thể cấu hình (mạng, máy chủ, lưu trữ, ứng dụng, dịch vụ), được cấp phát và giải phóng nhanh chóng với nỗ lực quản lý tối thiểu. Năm đặc trưng cốt lõi: tự phục vụ theo nhu cầu, truy cập mạng rộng rãi, gộp tài nguyên, co giãn nhanh và dịch vụ đo lường được.

*Bảng 2.1. Các mô hình dịch vụ và vị trí của Nimbus Drive*

| **Mô hình** | **Nhà cung cấp quản lý** | **Áp dụng trong đồ án** |
|---|---|---|
| IaaS | Phần cứng, ảo hóa, mạng | Amazon EC2, EBS, VPC, Elastic IP |
| PaaS / dịch vụ quản lý | Hệ điều hành, runtime, dịch vụ | Amazon S3 (object storage), GHCR, GitHub Actions |
| SaaS | Toàn bộ ứng dụng | Nimbus Drive cung cấp cho người dùng cuối dưới dạng SaaS |

## 2.2. Lưu trữ đối tượng và giao thức Amazon S3

Lưu trữ đối tượng tổ chức dữ liệu thành các **object** nằm phẳng trong **bucket**, mỗi object gồm dữ liệu, metadata và một **key** duy nhất. Khác với file system phân cấp hay block storage, object storage truy cập qua HTTP API, mở rộng ngang dễ dàng và có độ bền rất cao (Amazon S3 thiết kế cho độ bền 99,999999999%). Các thao tác chính: PutObject, GetObject, DeleteObject, HeadBucket, CreateBucket, ListObjectsV2.

Mọi request S3 được ký bằng **AWS Signature Version 4 (SigV4)** dựa trên access key/secret key. **Presigned URL** là URL đã được ký sẵn kèm thời hạn, cho phép bên thứ ba (ví dụ trình duyệt) tải trực tiếp object mà không cần credential, giúp giảm tải cho máy chủ ứng dụng. Vì S3 là chuẩn mở trên thực tế, ứng dụng viết theo S3 API có thể chuyển giữa AWS và các hệ thống tương thích như SeaweedFS, Ceph RGW, MinIO chỉ bằng cách đổi endpoint.

Trong quá trình thực hiện, nhóm ghi nhận **MinIO bản cộng đồng đã ngừng phát hành bản dựng sẵn** (trang tải về trả mã 410 Gone). Vì vậy đồ án chọn **SeaweedFS** (giấy phép Apache 2.0, vẫn được duy trì tích cực) làm S3 server tự host; nhờ thiết kế trừu tượng hóa, thay đổi này không ảnh hưởng mã nghiệp vụ – một minh chứng cho lợi ích của kiến trúc tương thích S3.

## 2.3. Container hóa với Docker

Container đóng gói ứng dụng cùng toàn bộ thư viện phụ thuộc thành **image** bất biến, chạy cô lập trên nhân hệ điều hành dùng chung nhờ namespaces và cgroups. So

với máy ảo, container khởi động nhanh, nhẹ và đảm bảo tính nhất quán "build once, run anywhere". **Docker Compose** mô tả hệ thống nhiều container (app, database, storage, proxy, giám sát) trong một file YAML, khai báo mạng, volume, healthcheck và thứ tự phụ thuộc.

## 2.4. Tích hợp và triển khai liên tục (CI/CD)

**CI** (Continuous Integration) là việc tự động build và kiểm thử mỗi khi có thay đổi mã nguồn để phát hiện lỗi sớm. **CD** (Continuous Delivery/Deployment) tự động đóng gói và đưa phiên bản đã kiểm thử lên môi trường chạy thật. GitHub Actions cung cấp runner được quản lý, định nghĩa pipeline bằng YAML theo sự kiện (push, pull request) và tích hợp sẵn registry GHCR.

## 2.5. Hạ tầng dưới dạng mã (Infrastructure as Code)

IaC mô tả hạ tầng (máy chủ, mạng, bucket, quyền) bằng mã nguồn có thể review, version và tái tạo. **Terraform** dùng ngôn ngữ khai báo HCL, so sánh trạng thái mong muốn với trạng thái thực tế để lập kế hoạch (plan) và áp dụng (apply) thay đổi.

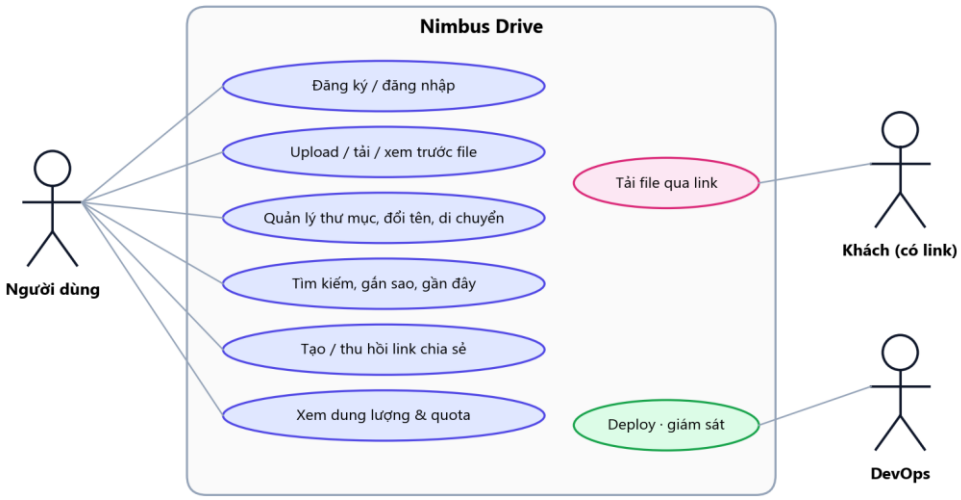
## 2.6. Các công nghệ phía ứng dụng

*Bảng 2.2. Công nghệ sử dụng*

| Thành phần | Công nghệ | Vai trò |
|---|---|---|
| Ngôn ngữ / framework | Java 21, Spring Boot 3.3 | REST API, DI, cấu hình theo môi trường |
| Bảo mật | Spring Security, JJWT 0.12, BCrypt | Xác thực stateless bằng JWT, băm mật khẩu |
| Dữ liệu | Spring Data JPA, Hibernate 6, PostgreSQL 16, Flyway | Lưu metadata, quản lý phiên bản schema |
| Lưu trữ | AWS SDK for Java v2 (S3Client, S3Presigner) | Giao tiếp S3 API, ký presigned URL |
| Tài liệu API | springdoc-openapi 2.6 (Swagger UI) | Tài liệu và thử API trực tuyến |
| Giám sát | Actuator, Micrometer, Prometheus, Grafana | Health check, metrics, dashboard |
| Kiểm thử | JUnit 5, MockMvc, Testcontainers, JaCoCo | Unit/API/integration test, đo độ phủ |
| Frontend | HTML5, CSS3, JavaScript thuần (ES2020) | SPA không cần bước build, tuân thủ CSP |
| Hạ tầng | Docker, Compose, Caddy, Terraform, GitHub Actions | Đóng gói, HTTPS, IaC, CI/CD |

# CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

## 3.1. Yêu cầu chức năng




*Hình 3.1. Sơ đồ use case tổng quát*

*Bảng 3.1. Yêu cầu chức năng*

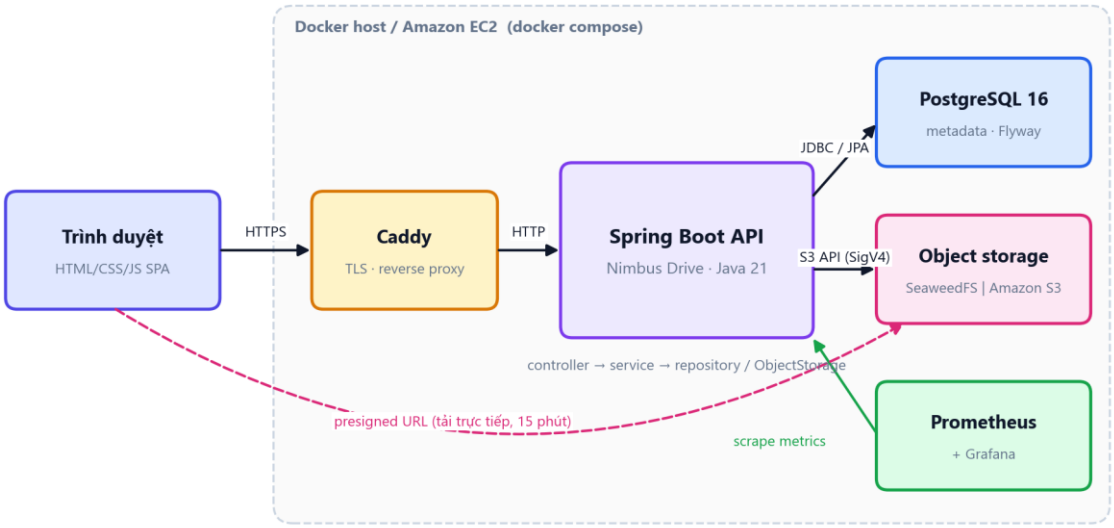
| Mã | Chức năng | Mô tả |
|---|---|---|
| F01 | Đăng ký / đăng nhập | Tên đăng nhập 3–50 ký tự [A-Za-z0-9._-], mật khẩu ≥ 6 ký tự; trả về JWT hạn 24 giờ |
| F02 | Upload file | Kéo-thả hoặc chọn nhiều file, hiển thị tiến độ; giới hạn 100 MB/file, kiểm tra quota |
| F03 | Tải xuống / xem trước | Xem ảnh, PDF, video, audio, văn bản/mã nguồn trên trình duyệt |
| F04 | Quản lý thư mục | Tạo, đổi tên, xóa đệ quy, thư mục lồng nhau, breadcrumb, di chuyển file |
| F05 | Tổ chức | Đổi tên, gắn sao, mục Gần đây, tìm kiếm theo tên trên toàn kho |
| F06 | Chia sẻ | Tạo link công khai có thời hạn, đếm lượt tải, thu hồi; trang tải cho khách |
| F07 | Presigned URL | Sinh URL S3 ký sẵn 15 phút để tải trực tiếp từ object storage |
| F08 | Dung lượng | Hiển thị dung lượng đã dùng / quota và phân bố theo loại file |

## 3.2. Yêu cầu phi chức năng

- **Bảo mật:** mật khẩu BCrypt; JWT ký HS256; khóa tài khoản tạm thời sau 5 lần đăng nhập sai; người dùng chỉ truy cập tài nguyên của mình (trả 404 thay vì 403 để không lộ sự tồn tại); CSP chặn script lạ; không phục vụ inline HTML/SVG do người dùng tải lên.
- **Khả năng di chuyển (portability):** chạy với AWS S3 hoặc S3-compatible bất kỳ; cấu hình hoàn toàn qua biến môi trường (12-factor).

- **Khả năng quan sát:** health/liveness/readiness probe, metrics Prometheus, metric nghiệp vụ (số file, dung lượng upload).
- **Tin cậy:** migration CSDL có phiên bản (Flyway); không để object mồ côi khi lỗi; tự thử lại khi storage khởi động chậm; graceful shutdown.
- **Khả dụng giao diện:** responsive trên điện thoại, chế độ sáng/tối, thông báo lỗi tiếng Việt rõ ràng.

## 3.3. Kiến trúc tổng thể

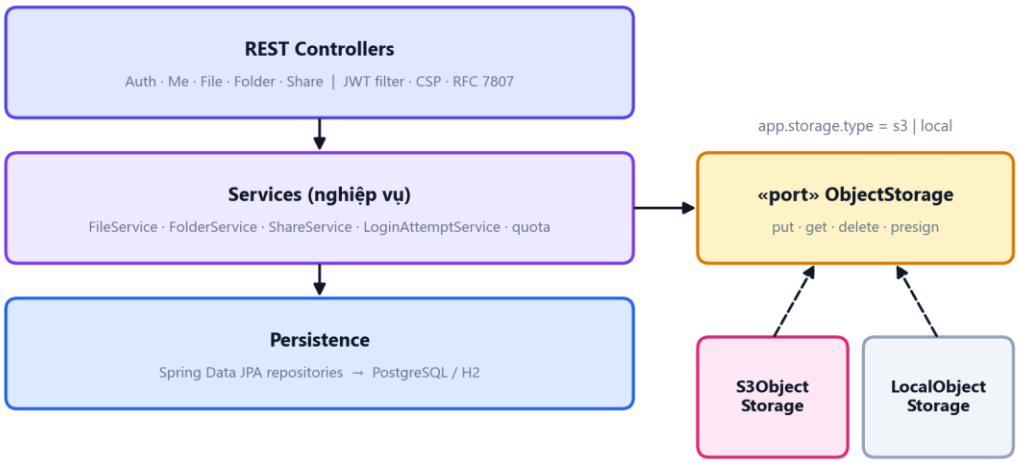



*Hình 3.2. Kiến trúc tổng thể hệ thống Nimbus Drive*

Hệ thống gồm các thành phần chạy trong container: **Caddy** làm reverse proxy và tự cấp chứng chỉ TLS (Let's Encrypt); **Spring Boot API** xử lý nghiệp vụ và phục vụ giao diện web tĩnh; **PostgreSQL** lưu người dùng, thư mục, metadata file và link chia sẻ; **object storage** lưu nội dung file. Khi chạy local/self-host, storage là SeaweedFS; khi chạy trên AWS là Amazon S3. Prometheus thu thập metrics và Grafana hiển thị dashboard (tùy chọn).

Việc tách metadata và nội dung là quyết định thiết kế quan trọng: CSDL quan hệ phù hợp cho truy vấn, phân quyền, tìm kiếm; object storage phù hợp cho dữ liệu nhị phân lớn, rẻ và bền. Thư mục là **thư mục ảo** chỉ tồn tại trong DB – object trên S3 được đặt key phẳng dạng owner/uuid, nên đổi tên hay di chuyển thư mục không cần sao chép dữ liệu trên storage.

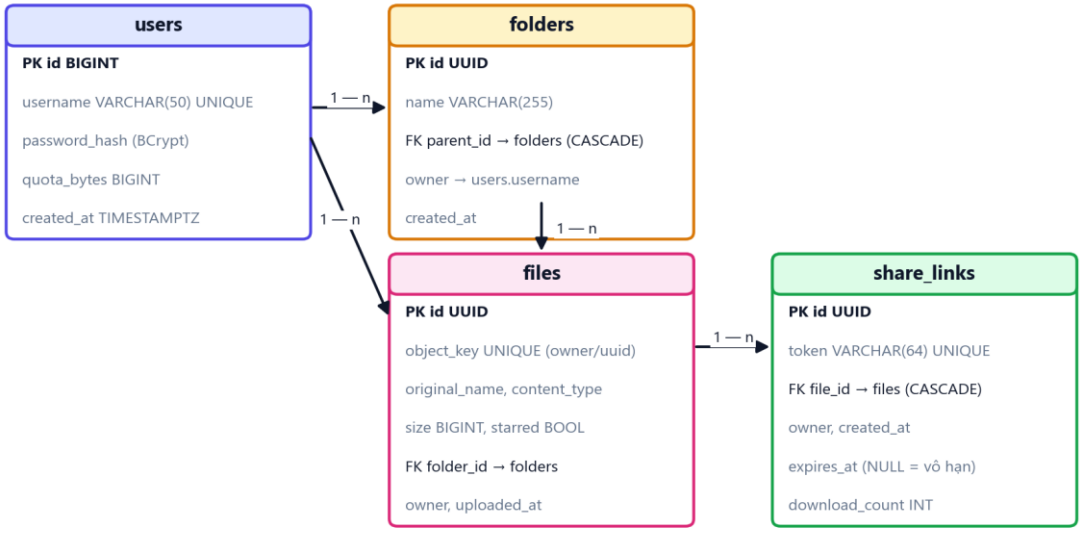
## 3.4. Kiến trúc phần mềm




*Hình 3.3. Kiến trúc phân lớp và mẫu Ports & Adapters cho tầng lưu trữ*

Mã nguồn chia lớp: Controller (REST, validate đầu vào) → Service (nghiệp vụ, kiểm tra quyền sở hữu, quota, transaction) → Repository (Spring Data JPA). Tầng lưu trữ được trừu tượng hóa bằng interface **ObjectStorage** (port) với hai adapter: **S3ObjectStorage** (AWS SDK v2) và **LocalObjectStorage** (ổ đĩa, dùng cho phát triển/kiểm thử). Adapter được chọn qua thuộc tính app.storage.type nhờ @ConditionalOnProperty.

## 3.5. Thiết kế cơ sở dữ liệu




*Hình 3.4. Lược đồ quan hệ thực thể (ERD)*

*Bảng 3.2. Mô tả các bảng*

| Bảng | Mục đích | Ràng buộc đáng chú ý |
|---|---|---|
| users | Tài khoản, mật khẩu băm, quota | username UNIQUE |
| folders | Cây thư mục ảo của từng người | parent_id tự tham chiếu, ON DELETE |

| | | |
|---|---|---|
| | dùng | CASCADE; index (owner, parent_id) |
| files | Metadata file, ánh xạ tới object key | object_key UNIQUE; index (owner, folder_id) |
| share_links | Link chia sẻ công khai | token UNIQUE (192 bit ngẫu nhiên); file_id ON DELETE CASCADE |

Schema được quản lý bằng **Flyway** (V1__init.sql); Hibernate đặt ddl-auto = none để tránh thay đổi schema ngầm định ở production. Cùng một script migration chạy được trên PostgreSQL (production) và H2 chế độ PostgreSQL (dev/test).

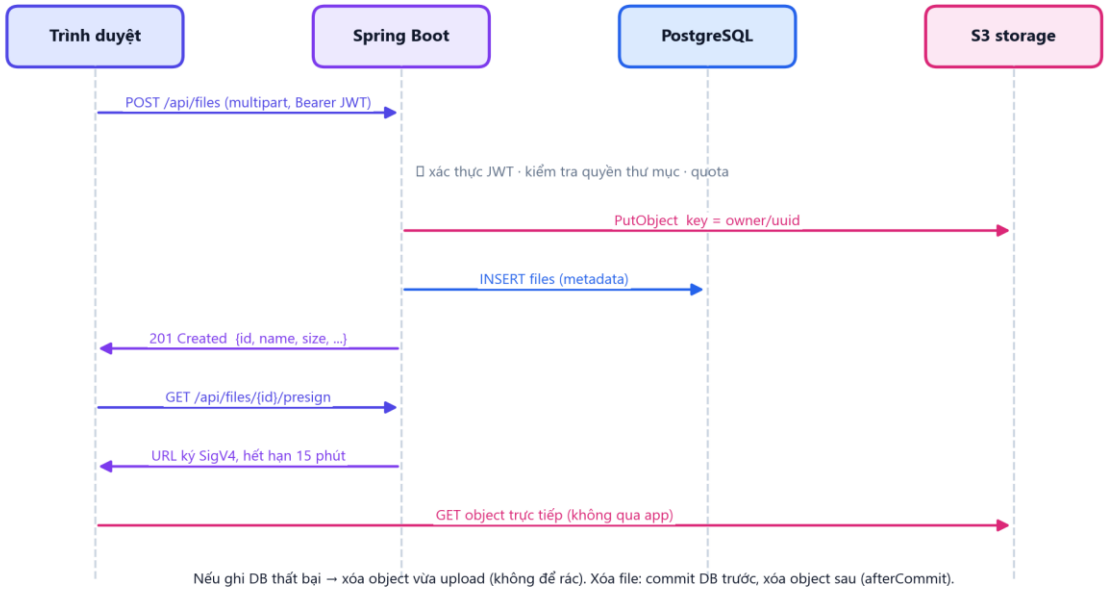
## 3.6. Thiết kế API

*Bảng 3.3. Các endpoint REST chính*

| Method | Endpoint | Mô tả |
|---|---|---|
| POST | /api/auth/register, /api/auth/login | Đăng ký / đăng nhập, trả JWT |
| GET | /api/me | Hồ sơ, dung lượng, thống kê theo loại |
| GET | /api/files?folderId=&q=&view= | Liệt kê theo thư mục, tìm kiếm, recent/starred |
| POST | /api/files (multipart) | Upload file vào thư mục |
| GET | /api/files/{id}/download?inline= | Tải hoặc xem trước |
| GET | /api/files/{id}/presign | Presigned URL S3 (15 phút) |
| PATCH / POST / PUT / DELETE | /api/files/{id}, /move, /star | Đổi tên, di chuyển, gắn sao, xóa |
| GET / POST / PATCH / DELETE | /api/folders, /api/folders/{id}/path | Quản lý thư mục, breadcrumb |
| POST | /api/files/{id}/shares | Tạo link chia sẻ có thời hạn |
| GET / DELETE | /api/shares, /api/shares/{id} | Danh sách / thu hồi link |
| GET | /api/public/shares/{token}[/download] | Công khai: thông tin và tải file |

Lỗi được trả về theo chuẩn **RFC 7807 Problem Details** (application/problem+json) với thông điệp tiếng Việt, ví dụ 413 "Vượt quá dung lượng cho phép", 429 khi đăng nhập sai quá nhiều, 410 khi link chia sẻ hết hạn, 502 khi không kết nối được storage.

## 3.7. Luồng xử lý upload và tải qua presigned URL




*Hình 3.5. Sơ đồ tuần tự upload file và tải trực tiếp bằng presigned URL*

Khi upload, service kiểm tra JWT, quyền với thư mục đích và quota, sau đó ghi object lên S3 trước rồi mới ghi metadata. Nếu ghi DB thất bại, object vừa ghi bị xóa để không để lại dữ liệu mồ côi. Khi xóa, metadata được xóa trong transaction, còn object chỉ bị xóa **sau khi commit** thành công (TransactionSynchronization.afterCommit) – tránh trường hợp mất dữ liệu thật nhưng metadata vẫn còn.

## 3.8. Thiết kế bảo mật

*Bảng 3.4. Các biện pháp bảo mật*

| Mối đe dọa | Biện pháp |
|---|---|
| Lộ mật khẩu | BCrypt (cost 10); không bao giờ trả về hash |
| Dò mật khẩu (brute force) | LoginAttemptService khóa 5 phút sau 5 lần sai liên tiếp (HTTP 429) |
| Truy cập trái phép (IDOR) | Mọi truy vấn lọc theo owner; tài nguyên của người khác → 404 |
| XSS qua file tải lên | Chỉ hiển thị inline ảnh raster/PDF/text/video/audio; HTML, SVG… ép tải về octet-stream; header CSP sandbox trên response file |
| XSS trên giao diện | CSP script-src 'self', không inline script/handler; escape mọi dữ liệu khi render |
| Path traversal | Làm sạch tên file (bỏ ../, ký tự điều khiển); LocalObjectStorage kiểm tra key nằm trong thư mục gốc |
| Đoán link chia sẻ | Token 24 byte SecureRandom (192 bit), Base64URL; có thời hạn và thu hồi được |
| Lộ credential S3 trên AWS | Dùng IAM instance profile, không lưu access key; IMDSv2 bắt buộc; policy chỉ Get/Put/Delete trên 1 bucket |
| Lộ dữ liệu trên S3 | Block Public Access, mã hóa SSE-S3, versioning, |

| | |
|---|---|
| | BucketOwnerEnforced |
| Nghe lén | HTTPS qua Caddy (Let’s Encrypt), app chỉ lắng nghe 127.0.0.1 trên EC2 |

# CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

## 4.1. Cấu trúc mã nguồn

```
cloud-file-storage/
├── src/main/java/com/cloudfs/
│   ├── config/        StorageConfig (S3Client, S3Presigner), OpenApiConfig
│   ├── controller/    Auth, Me, File, Folder, Share
│   ├── service/       FileService, FolderService, ShareService, LoginAttemptService
│   ├── storage/       ObjectStorage, S3ObjectStorage, LocalObjectStorage
│   ├── security/      JwtService, JwtAuthFilter, SecurityConfig
│   └── web/           ApiExceptionHandler (RFC 7807)
├── src/main/resources/
│   ├── db/migration/V1__init.sql
│   └── static/        index.html, app.html, share.html, css/, js/
├── src/test/          ApiFlowTest, UnitTests, *IT (Testcontainers)
├── deploy/            seaweedfs/, caddy/, monitoring/, terraform/
├── Dockerfile, docker-compose.yml, docker-compose.aws.yml
└── .github/workflows/ci-cd.yml
```

## 4.2. Tầng lưu trữ tương thích S3

StorageConfig tạo S3Client và S3Presigner từ cấu hình. Khi S3_ENDPOINT rỗng, SDK dùng endpoint AWS thật; khi có giá trị, ứng dụng kết nối tới storage tương thích (SeaweedFS) với path-style addressing. Nếu không khai báo access key, ứng dụng dùng **DefaultCredentialsProvider** – trên EC2 tự lấy credential tạm thời từ IAM role. Presigner có thể dùng endpoint công khai riêng (S3_PUBLIC_ENDPOINT) vì URL ký sẵn phải trỏ tới địa chỉ mà trình duyệt truy cập được.

```
public interface ObjectStorage {
    void put(String key, InputStream content, long size, String contentType) throws
IOException;
    InputStream get(String key) throws IOException;
    void delete(String key);
    Optional<URL> presignDownload(String key, String fileName, Duration ttl);
    String backendName();
}

// S3ObjectStorage.presignDownload
var request = GetObjectRequest.builder().bucket(bucket).key(key)

.responseContentDisposition(ContentDispositions.attachment(fileName)).build();
return Optional.of(presigner.presignGetObject(p -> p.signatureDuration(ttl)
        .getObjectRequest(request)).url());
```

Khi khởi động, S3ObjectStorage.ensureBucket() kiểm tra (HeadBucket) và tạo bucket nếu chưa có, thử lại tối đa 10 lần cách nhau 3 giây để chịu được việc storage khởi động chậm hơn ứng dụng trong Docker Compose.

## 4.3. Giao diện người dùng

Giao diện là SPA viết bằng JavaScript thuần, không cần bước build, được Spring Boot phục vụ dưới dạng tài nguyên tĩnh. Các thành phần chính: trang đăng nhập/đăng ký, trang quản lý file (sidebar điều hướng, thanh tìm kiếm, breadcrumb, lưới/danh sách, menu ngữ cảnh, hộp thoại xem trước, chia sẻ, di chuyển), bảng tiến độ upload và trang tải công khai. Upload dùng XMLHttpRequest để có sự kiện tiến độ; ảnh thu nhỏ và bản xem trước được tải bằng fetch kèm JWT rồi hiển thị qua blob URL.

## 4.4. Container hóa

### 4.4.1. Dockerfile nhiều giai đoạn

```
FROM maven:3.9-eclipse-temurin-21 AS build
RUN --mount=type=cache,target=/root/.m2 mvn -B -q dependency:go-offline
RUN --mount=type=cache,target=/root/.m2 mvn -B -q package -DskipTests \
    && java -Djarmode=tools -jar target/*.jar extract --layers --launcher --
destination extracted

FROM eclipse-temurin:21-jre
RUN useradd --system --uid 10001 appuser
COPY --from=build /workspace/extracted/dependencies/ ./
COPY --from=build /workspace/extracted/application/ ./
USER appuser
HEALTHCHECK CMD curl -fs http://localhost:8080/actuator/health/liveness || exit 1
ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS
org.springframework.boot.loader.launch.JarLauncher"]
```

- Giai đoạn build dùng JDK + Maven; image chạy chỉ chứa JRE → nhỏ gọn, ít bề mặt tấn công.
- Jar được tách **layer** (dependencies, loader, application): khi chỉ sửa mã, Docker chỉ build lại layer application.
- Chạy bằng user thường (UID 10001), có HEALTHCHECK, JVM giới hạn theo bộ nhớ container (MaxRAMPercentage=75).

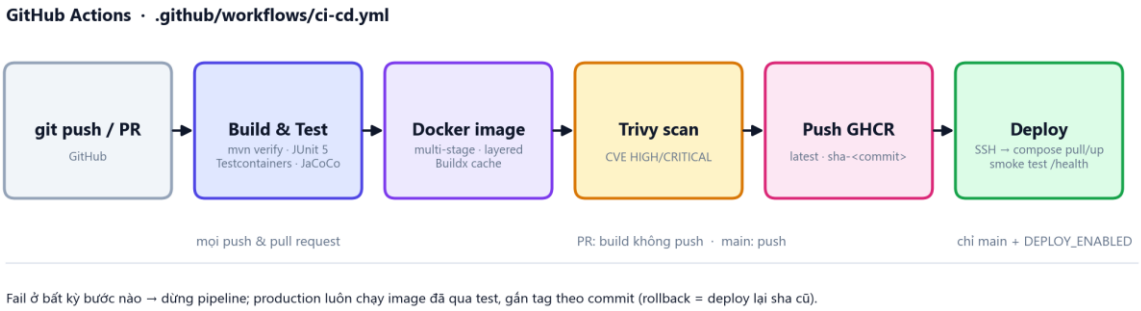
### 4.4.2. Docker Compose

*Bảng 4.1. Các service trong docker-compose.yml*

| Service | Image | Vai trò |
|---|---|---|
| postgres | postgres:16-alpine | CSDL metadata, volume pgdata, healthcheck pg_isready |
| seaweedfs | chrislusf/seaweedfs:4.52 | S3 server tự host, cổng 8333; danh tính S3 sinh từ biến môi trường |
| app | nimbus-drive (build từ Dockerfile) | Spring Boot; chờ postgres và seaweedfs healthy |
| prometheus, grafana | profile monitoring | Thu thập và hiển thị metrics, dashboard được provision sẵn |
| caddy | profile proxy | Reverse proxy HTTPS tự động, chặn public /actuator/prometheus |

File docker-compose.aws.yml ghi đè cho môi trường AWS: tắt SeaweedFS, để trống endpoint và access key (dùng Amazon S3 thật qua IAM role), và chỉ mở cổng app trên 127.0.0.1 để mọi truy cập bên ngoài phải đi qua Caddy.

## 4.5. Pipeline CI/CD với GitHub Actions



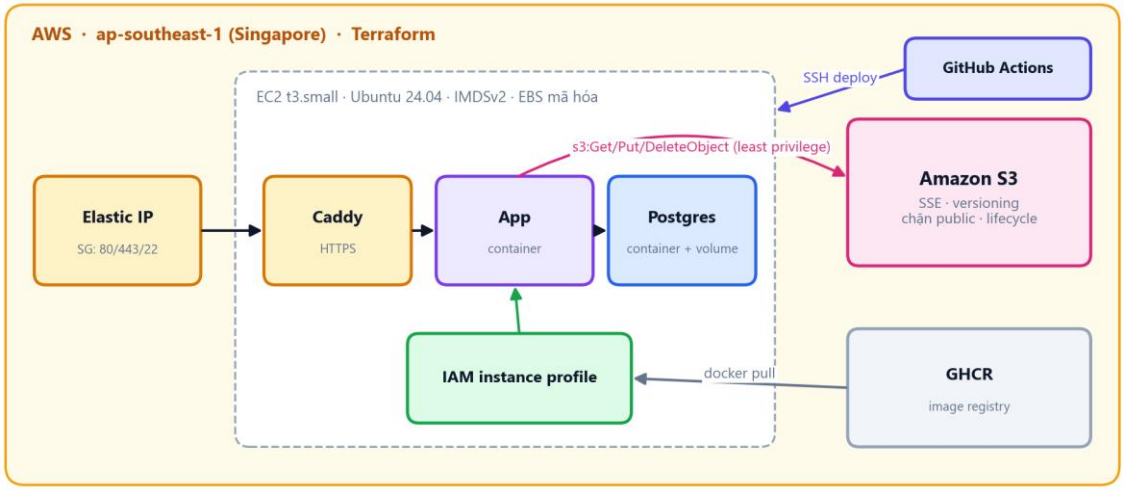

*Hình 4.1. Pipeline CI/CD tự động*

*Bảng 4.2. Các job trong pipeline*

| Job | Kích hoạt | Nội dung |
|---|---|---|
| test | Mọi push và pull request | mvn verify: unit test, API test, integration test bằng Testcontainers (PostgreSQL, SeaweedFS); lưu báo cáo test và độ phủ JaCoCo |
| image | Sau test | Docker Buildx với cache GHA; PR chỉ build kiểm tra, main thì push GHCR tag latest + sha-<commit>; quét Trivy lỗ hổng HIGH/CRITICAL |
| deploy | Push main, DEPLOY_ENABLED=true | SCP file compose lên server, docker compose pull/up với image đúng commit, smoke test /actuator/health trong 90 giây, in log nếu thất bại |

Image production được gắn tag theo commit nên có thể truy vết và **rollback** bằng cách triển khai lại tag cũ. Job deploy gắn với GitHub Environment "production", cho phép bật duyệt thủ công (required reviewers) nếu cần.

## 4.6. Hạ tầng AWS bằng Terraform




*Hình 4.2. Kiến trúc triển khai trên AWS*

*Bảng 4.3. Tài nguyên Terraform tạo ra*

| Tài nguyên | Cấu hình |
|---|---|
| aws_s3_bucket + cấu hình | Block Public Access, SSE-AES256, versioning, lifecycle (xóa phiên bản cũ sau 30 ngày, hủy multipart dở sau 7 ngày), CORS cho presigned GET |
| aws_iam_role / instance_profile | Chỉ s3:ListBucket trên bucket và s3:Get/Put/DeleteObject trên bucket/*; kèm AmazonSSMManagedInstanceCore |
| aws_security_group | Mở 80, 443 công khai; 22 theo danh sách CIDR |
| aws_instance | Ubuntu 24.04, t3.small, IMDSv2 bắt buộc, EBS gp3 mã hóa; user_data cài Docker và sinh file .env (mật khẩu DB, JWT secret ngẫu nhiên) |
| aws_eip | Địa chỉ IP tĩnh để trỏ DNS và làm DEPLOY_HOST |

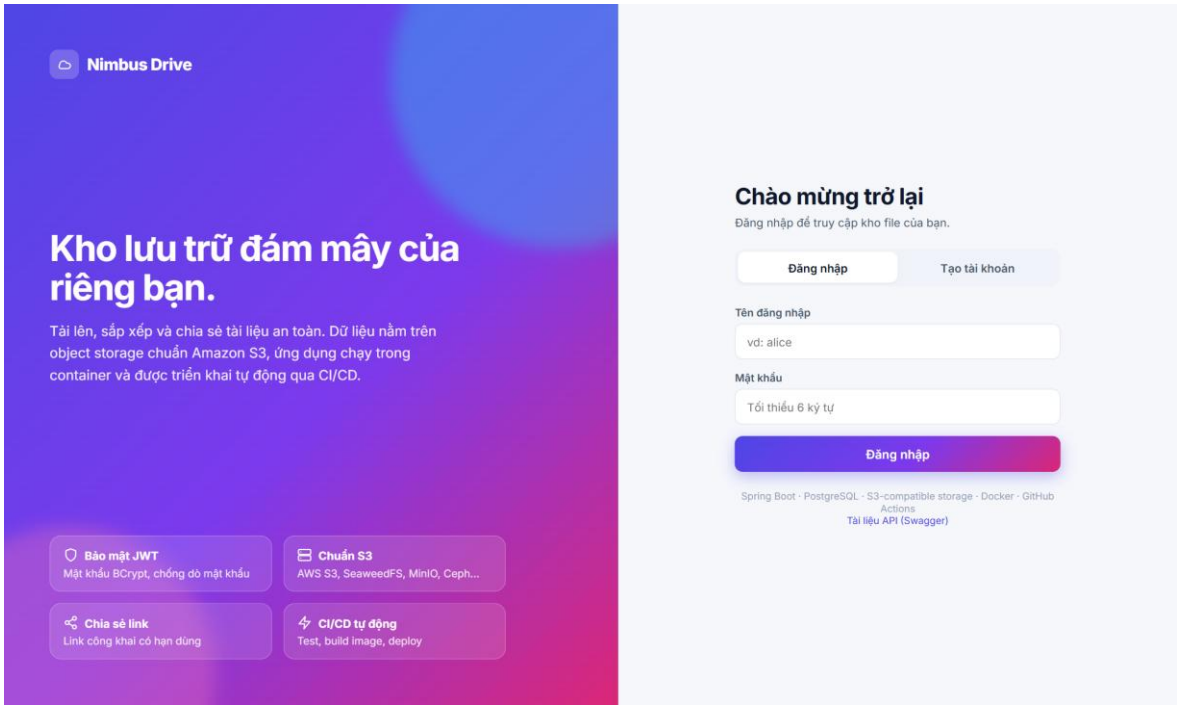
```
cd deploy/terraform
cp terraform.tfvars.example terraform.tfvars   # key_name, app_image, domain
terraform init
terraform plan
terraform apply        # output: public_ip, bucket, app_url
```

## 4.7. Giám sát

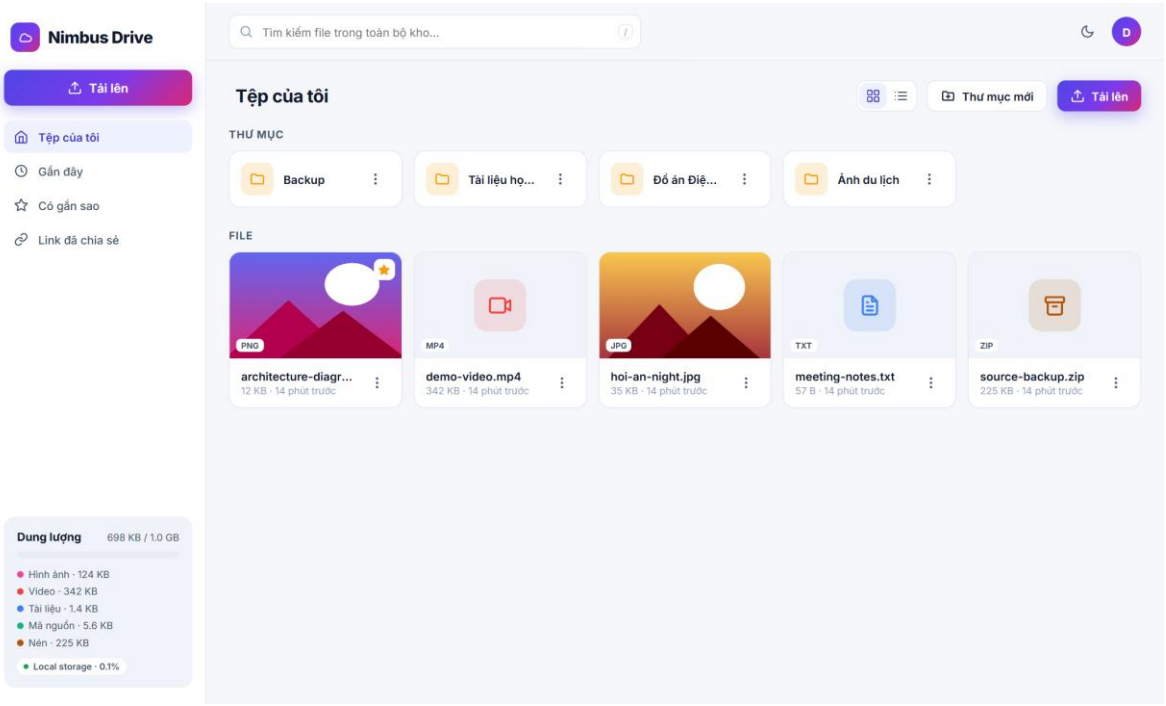
Spring Boot Actuator cung cấp /actuator/health (kèm liveness/readiness), /actuator/prometheus. Ngoài các metric có sẵn (HTTP request, độ trễ p95, JVM heap, kết nối Hikari), ứng dụng đăng ký metric nghiệp vụ nimbus.files.uploaded và nimbus.files.upload.bytes. Dashboard Grafana "Nimbus Drive – Overview" được provision tự động khi bật profile monitoring.

# CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

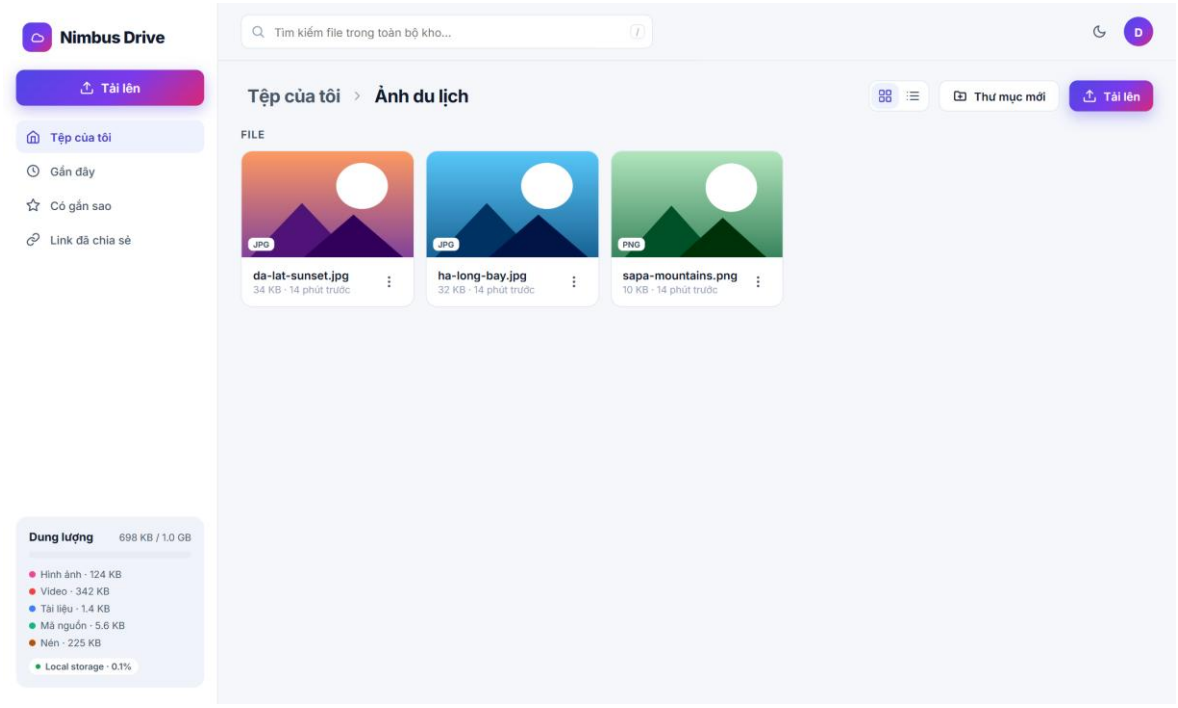
## 5.1. Giao diện ứng dụng



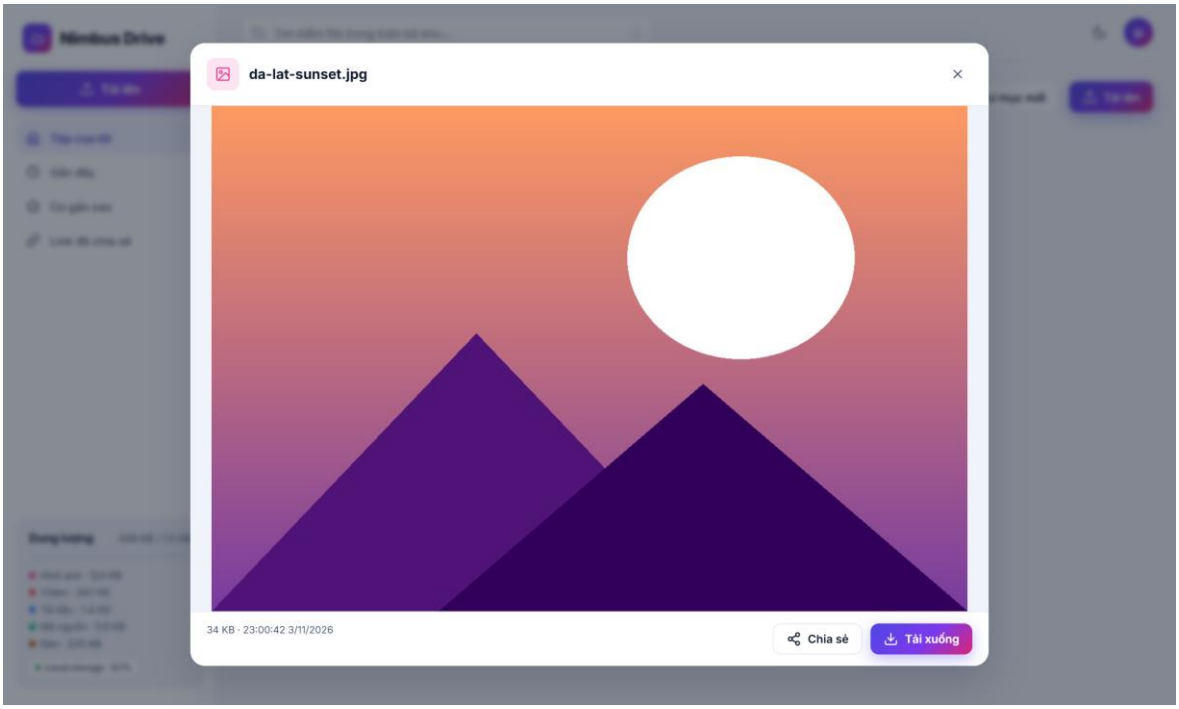

*Hình 5.1. Trang đăng nhập / đăng ký*



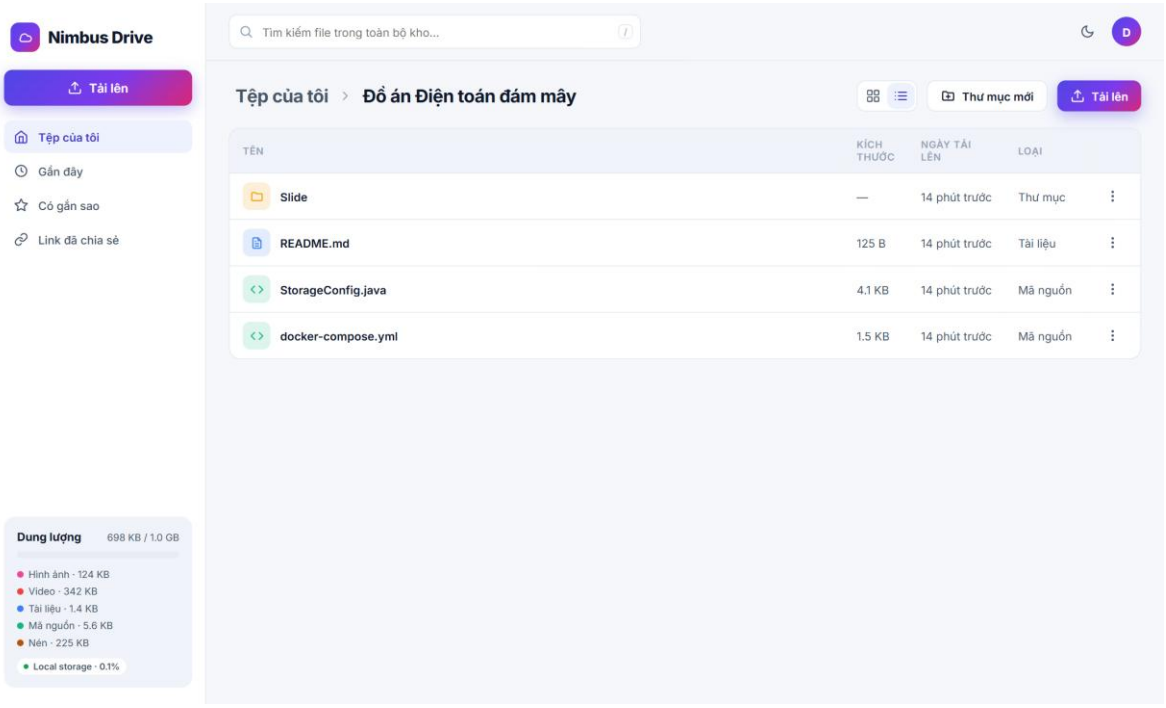

*Hình 5.2. Trang quản lý file – chế độ lưới*



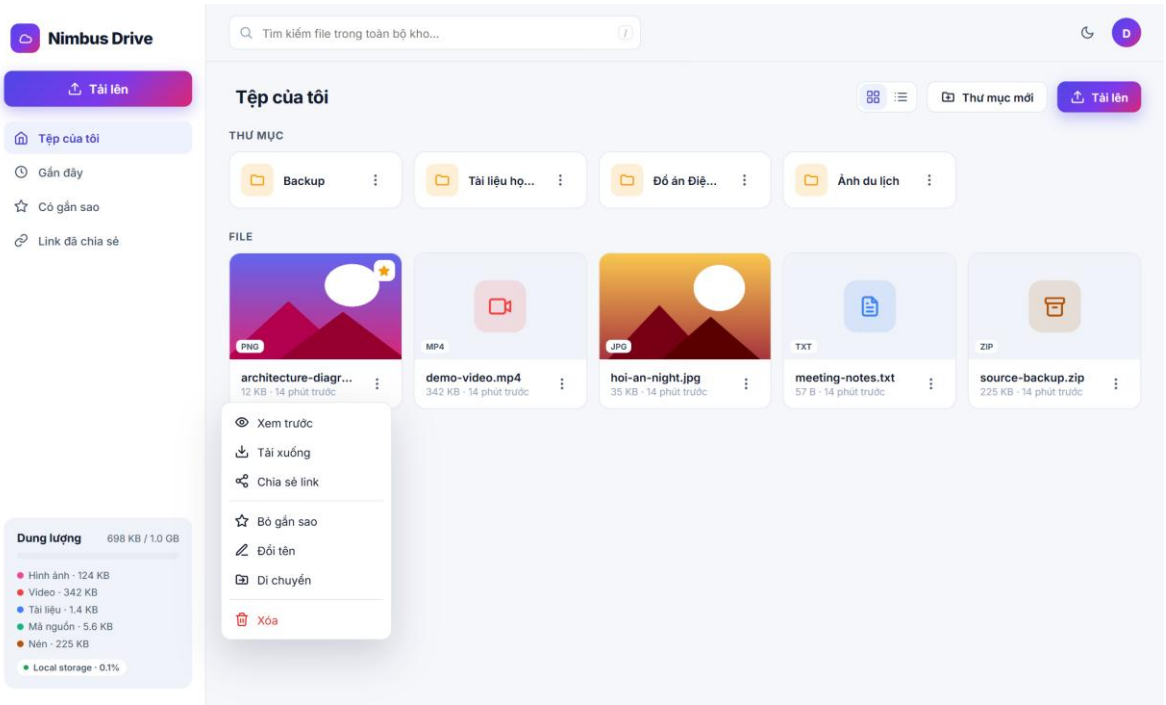

*Hình 5.3. Thư mục ảnh với ảnh thu nhỏ*



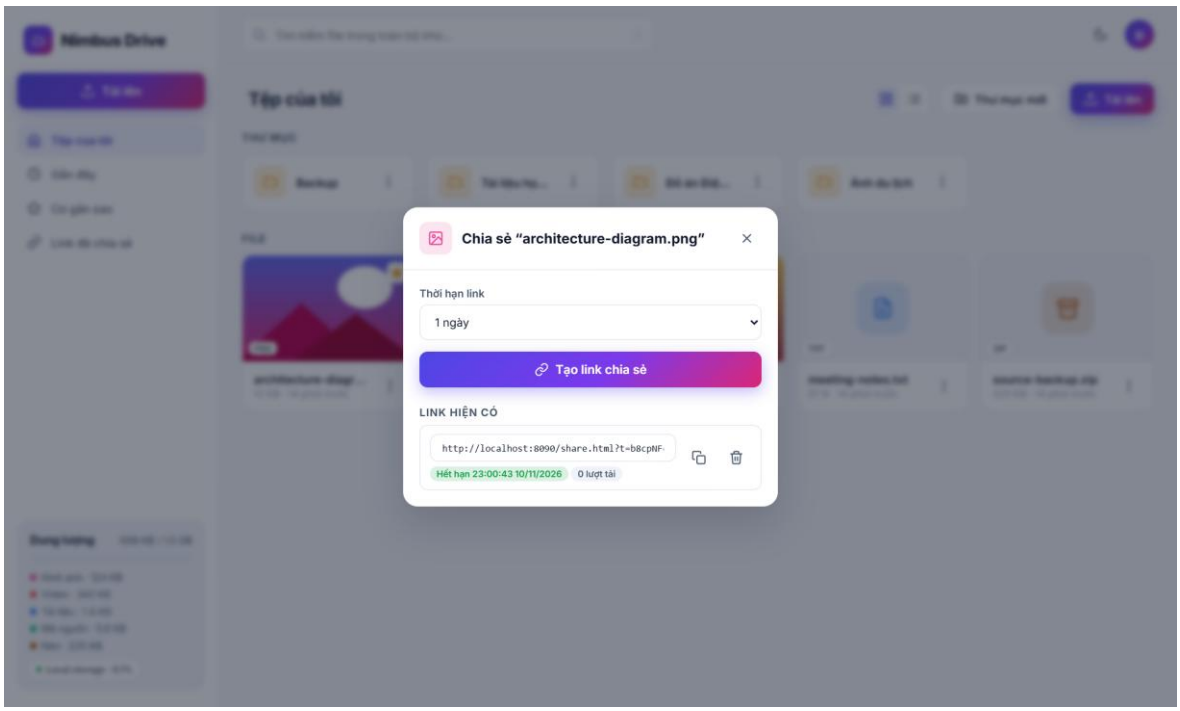

*Hình 5.4. Xem trước ảnh trực tiếp*



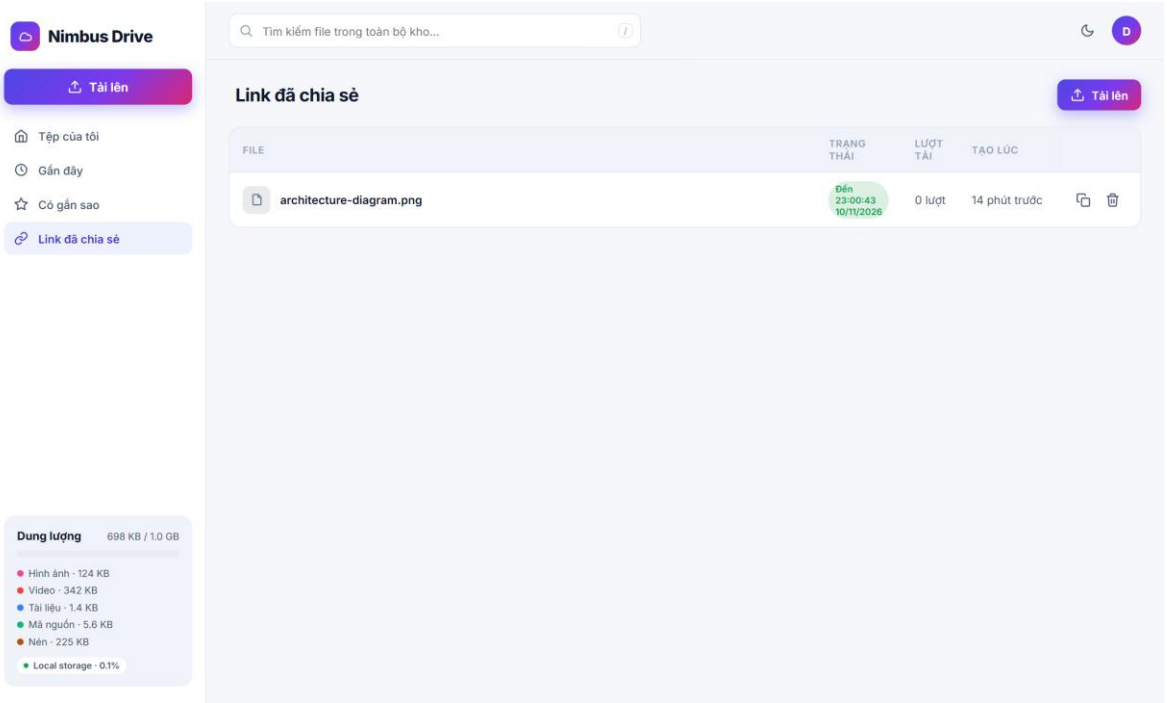

*Hình 5.5. Chế độ danh sách và breadcrumb thư mục lồng nhau*



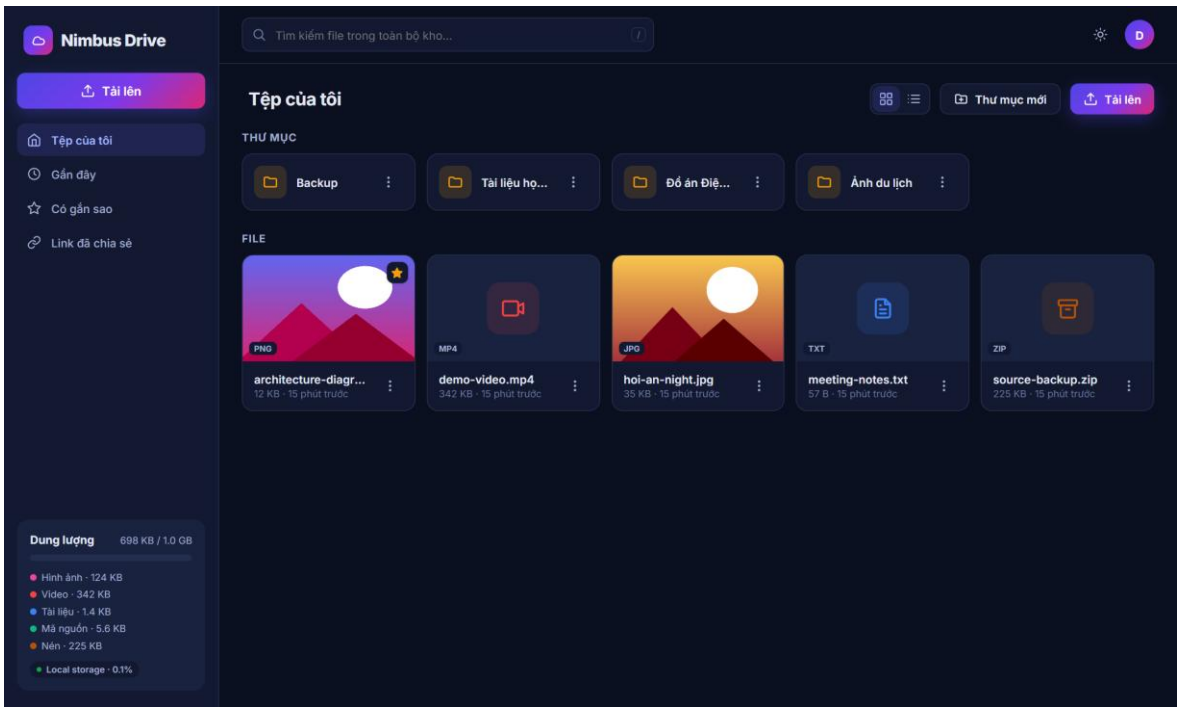

*Hình 5.6. Menu thao tác trên file*



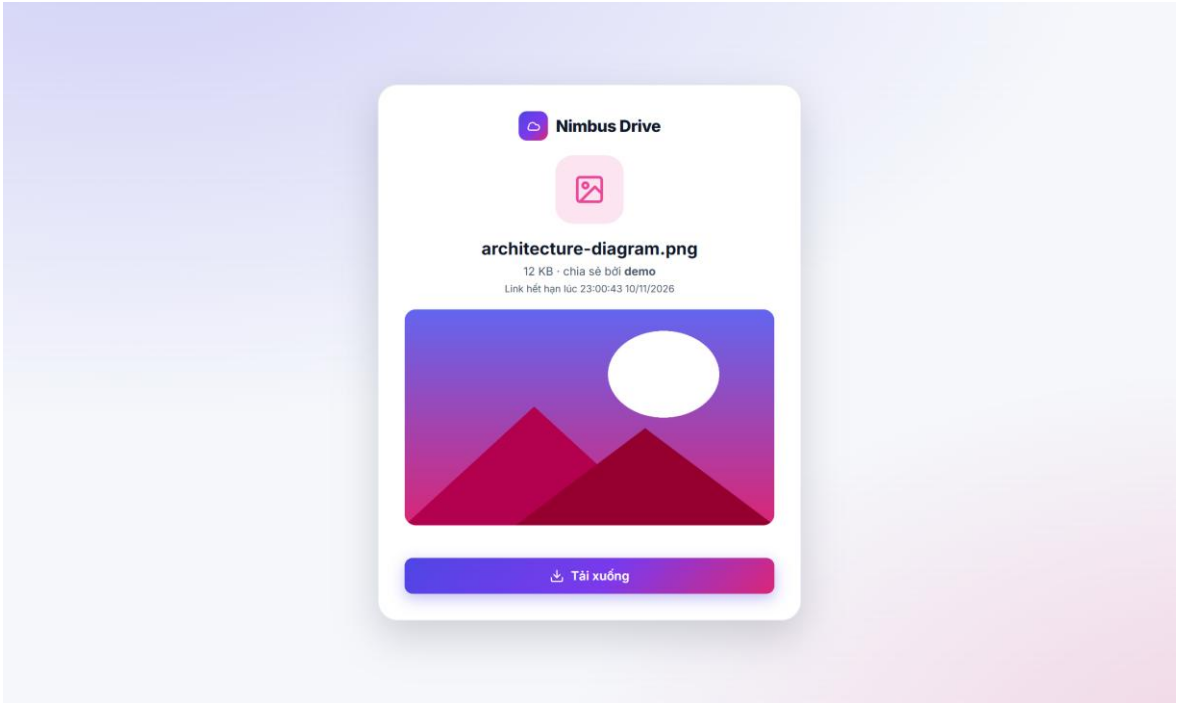

*Hình 5.7. Hộp thoại tạo link chia sẻ có thời hạn*



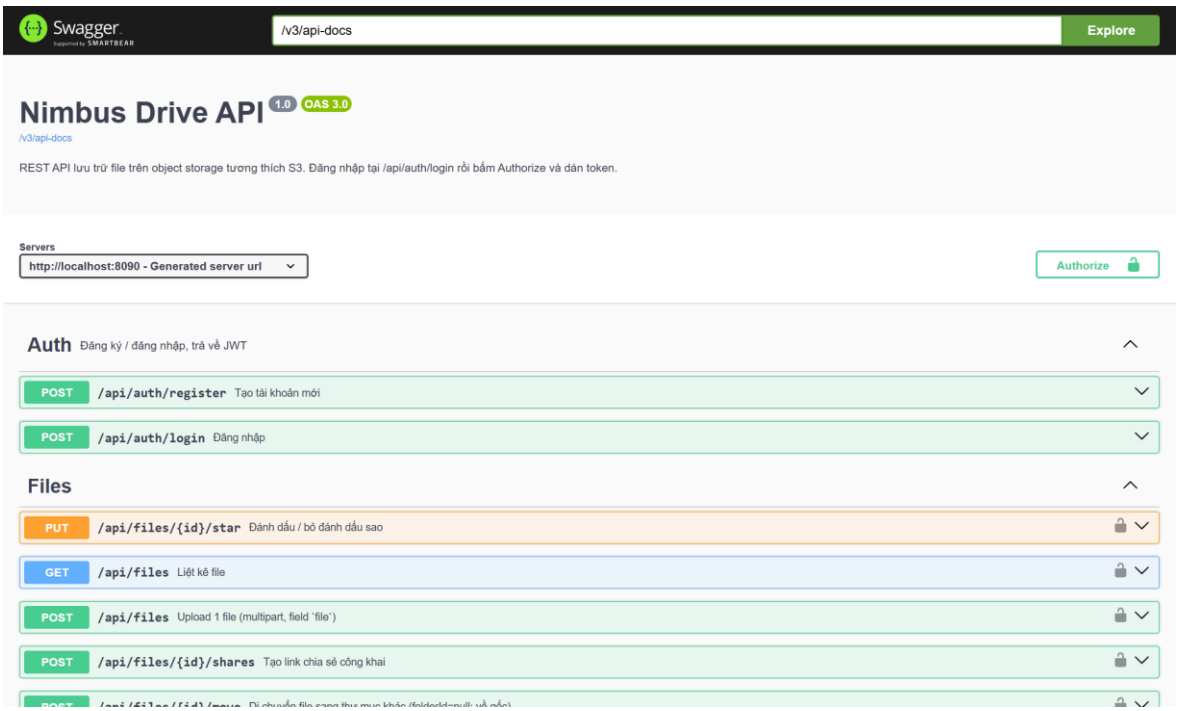

*Hình 5.8. Quản lý các link đã chia sẻ*



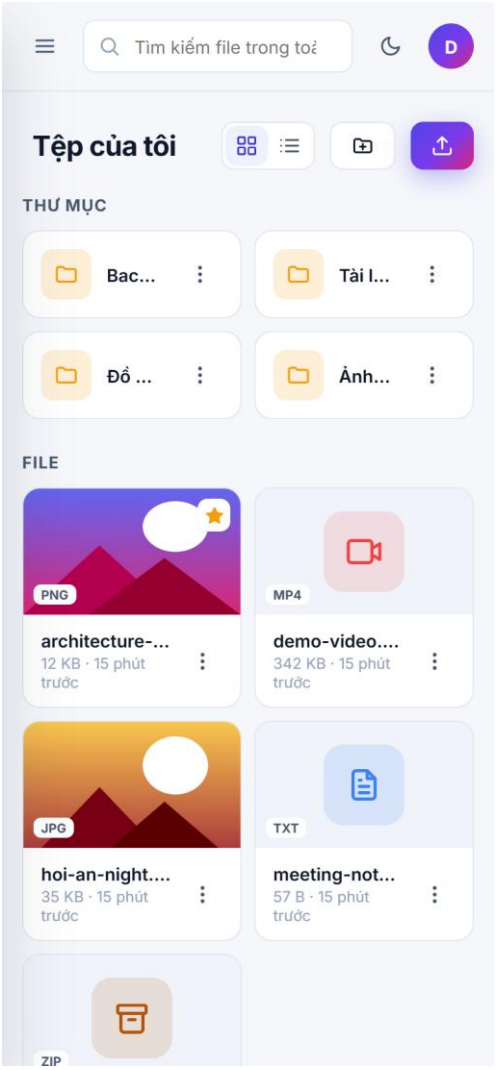

*Hình 5.9. Giao diện tối (dark mode)*




*Hình 5.10. Trang tải file công khai cho khách*




*Hình 5.11. Tài liệu API Swagger UI*




*Hình 5.12. Giao diện trên điện thoại*

## 5.2. Kiểm thử

*Bảng 5.1. Kết quả kiểm thử tự động (mvn verify)*

| Bộ test | Số ca | Nội dung | Kết quả |
|---|---|---|---|
| JwtServiceTest | 3 | Sinh/giải mã token, sai khóa, hết hạn | Đạt |
| UnitTests | 4 | Làm sạch tên file, phân loại file, token chia sẻ, khóa đăng nhập | Đạt |
| ApiFlowTest (MockMvc + H2) | 11 | Bảo vệ endpoint, validate, trùng tài khoản, khóa đăng nhập, upload/tải/đổi tên/gắn sao/xóa, cô lập người dùng, quota, thư mục đệ quy, chia sẻ, chống XSS, presign, health | Đạt |
| PostgresMigrationIT | 1 | Flyway + JPA trên PostgreSQL 16 thật (Testcontainers) | Chạy trên CI* |
| S3ObjectStorageIT | 3 | Put/Get/Delete, presigned URL, tạo bucket idempotent trên SeaweedFS thật | Chạy trên CI* |

18/18 ca kiểm thử chạy cục bộ đều đạt; **độ phủ dòng lệnh đạt khoảng 80%** (JaCoCo). (*) Các bài integration test cần Docker nên được cấu hình tự bỏ qua trên máy không có Docker và chạy trên runner GitHub Actions.

## 5.3. Đánh giá

*Bảng 5.2. Đối chiếu mục tiêu*

| Mục tiêu | Kết quả |
|---|---|
| Ứng dụng lưu trữ file đầy đủ chức năng | Hoàn thành F01–F08, giao diện responsive, sáng/tối |
| Kiến trúc tương thích S3 | Chạy với SeaweedFS (local) và Amazon S3 (AWS) không đổi mã; hỗ trợ presigned URL |
| Container hóa | Dockerfile multi-stage, layered, non-root; Compose 5 service với healthcheck và profile |
| CI/CD tự động | Pipeline test → image → scan → push → deploy → smoke test |
| IaC trên AWS | Terraform cho S3, IAM, EC2, SG, EIP theo nguyên tắc quyền tối thiểu |
| Bảo mật, giám sát | JWT, BCrypt, rate limit, CSP, chống IDOR/XSS; Actuator, Prometheus, Grafana |

### Ưu điểm

- Tách biệt rõ metadata và nội dung, thư mục ảo giúp thao tác tổ chức file không tốn chi phí trên storage.
- Không phụ thuộc nhà cung cấp (vendor-neutral) nhờ port/adapter và chuẩn S3 – đã chứng minh khi thay MinIO bằng SeaweedFS.

- Có thể chạy ở ba chế độ: dev không Docker (H2 + ổ đĩa), Docker Compose self-host, AWS.
- Quy trình phát hành tự động, có kiểm thử, quét bảo mật và khả năng rollback theo commit.

### Hạn chế

- Upload/tải trong giao diện đi qua máy chủ ứng dụng (giới hạn 100 MB/file); chưa dùng multipart upload trực tiếp lên S3.
- Bộ đếm chống dò mật khẩu lưu trong RAM nên chưa dùng chung khi chạy nhiều instance.
- PostgreSQL chạy trong container trên cùng EC2 – một điểm lỗi đơn; chưa có backup tự động.
- Terraform đã viết và rà soát nhưng chưa được apply trên tài khoản AWS thật trong khuôn khổ đồ án.

# CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 6.1. Kết luận

Đồ án đã thiết kế và triển khai thành công **Nimbus Drive** – nền tảng lưu trữ file đám mây đóng gói container, xây dựng trên Spring Boot và kiến trúc tương thích Amazon S3, kèm pipeline CI/CD tự động hoàn chỉnh và hạ tầng AWS dưới dạng mã. Qua đó, em đã vận dụng các kiến thức cốt lõi của điện toán đám mây: mô hình IaaS/PaaS/SaaS, lưu trữ đối tượng, container hóa, quản lý định danh và truy cập (IAM), tự động hóa triển khai và giám sát hệ thống.

## 6.2. Hướng phát triển

*Bảng 6.1. Đề xuất phát triển tiếp*

| Hướng | Mô tả | Ưu tiên |
|---|---|---|
| Upload trực tiếp lên S3 | Presigned PUT + S3 multipart upload: file nhiều GB, giảm tải server, resume khi mất mạng | Cao |
| Thùng rác & phiên bản | Xóa mềm, khôi phục trong 30 ngày; tận dụng S3 versioning để xem/khôi phục phiên bản cũ | Cao |
| Hạ tầng managed | Amazon RDS (Multi-AZ, backup tự động), ECS Fargate/EKS + ALB, CloudFront CDN | Trung bình |
| Chia sẻ nội bộ | Chia sẻ cho user khác với quyền xem/sửa, link có mật khẩu, giới hạn lượt tải | Trung bình |
| Bảo mật nâng cao | Refresh token, đăng nhập Google (OAuth2/OIDC), 2FA, quét virus ClamAV, rate limit bằng Redis | Trung bình |
| Trải nghiệm | Upload cả thư mục, thumbnail sinh phía server (S3 event + Lambda), tìm kiếm toàn văn | Thấp |
| Vận hành | Cảnh báo Alertmanager, log tập trung (Loki/CloudWatch), staging environment, blue-green deploy | Thấp |

# TÀI LIỆU THAM KHẢO


[1] P. Mell, T. Grance, "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, 2011.

[2] Amazon Web Services, "Amazon S3 User Guide," docs.aws.amazon.com/AmazonS3.

[3] Amazon Web Services, "Authenticating Requests (AWS Signature Version 4)" và "Sharing objects with presigned URLs".

[4] Amazon Web Services, "AWS SDK for Java 2.x Developer Guide".

[5] VMware Tanzu, "Spring Boot Reference Documentation," phiên bản 3.3.

[6] Spring, "Spring Security Reference" – Authentication, Headers (Content Security Policy).

[7] Docker Inc., "Dockerfile reference" và "Docker Compose specification".

[8] GitHub, "GitHub Actions documentation" – Workflow syntax, Publishing Docker images.

[9] HashiCorp, "Terraform AWS Provider documentation".

[10] SeaweedFS, "Amazon S3 API" – github.com/seaweedfs/seaweedfs/wiki.

[11] M. Nottingham, E. Wilde, "RFC 7807 – Problem Details for HTTP APIs," IETF, 2016.

[12] OWASP Foundation, "OWASP Top 10" và "File Upload Cheat Sheet".

[13] A. Wiggins, "The Twelve-Factor App," 12factor.net.

# PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY

**A. Chạy bằng Docker (đầy đủ):**

```
cp .env.example .env          # đổi mật khẩu, JWT_SECRET
docker compose up -d --build
# Web: http://localhost:8080   Swagger: /swagger-ui.html   S3: http://localhost:8333
docker compose --profile monitoring up -d   # Grafana http://localhost:3000
```

**B. Chạy chế độ dev không cần Docker** (H2 + lưu file trên ổ đĩa):

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev     # Windows: mvnw.cmd
```

**C. Kiểm thử:**

```
./mvnw verify      # báo cáo độ phủ: target/site/jacoco/index.html
```

**D. Bật triển khai tự động:** tạo repo variable DEPLOY_ENABLED=true, APP_URL; secrets DEPLOY_HOST, DEPLOY_USER, DEPLOY_SSH_KEY; sau đó mỗi lần push lên nhánh main, pipeline tự build, kiểm thử và triển khai.